

PART 1 — Start the Server Automatically at Reboot (systemd)

1 Decide where the server lives

Best practice on RHEL:

```
/opt/east-side-server/
```

Example:

```
sudo mkdir -p /opt/east-side-server
sudo cp -r east-side-server/* /opt/east-side-server/
sudo chown -R root:root /opt/east-side-server
```

Your main file:

```
/opt/east-side-server/server.py
```

2 Confirm Python path (important)

On RHEL 9/10 this is usually:

```
/usr/bin/python3
```

Verify:

```
which python3
```

3 Create a systemd service file

```
sudo nano /etc/systemd/system/east-side-server.service
```

Paste **exactly this**:

```
[Unit]
Description=East Side Server (Upload + Routing Service)
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/east-side-server/server.py
WorkingDirectory=/opt/east-side-server
```

```
Restart=always
RestartSec=5
User=root
Group=root
Environment=PYTHONUNBUFFERED=1
```

```
[Install]
WantedBy=multi-user.target
```

Why this is correct

- `After=network.target` → ensures networking is ready
 - `Restart=always` → auto-recovery
 - `WorkingDirectory` → relative paths work
 - `PYTHONUNBUFFERED` → logs show immediately
-

4 Reload systemd and enable the service

```
sudo systemctl daemon-reload
sudo systemctl enable east-side-server
```

5 Start it now (first run)

```
sudo systemctl start east-side-server
```

Verify:

```
sudo systemctl status east-side-server
```

Expected:

```
Active: active (running)
```

6 Reboot test (important)

```
sudo reboot
```

After reboot:

```
sudo systemctl status east-side-server
```

- ✓ Server running automatically
 - ✓ Port 8000 listening
-

PART 2 — Control It From Another Dashboard (Button-Based)

You now have **two clean options**, depending on how much control you want.

OPTION A (Recommended): Dashboard Button → HTTP Health Check

This is the **safe, normal approach**.

What the button does

- Opens the server URL
- Confirms it is running
- No privilege escalation needed

Example button action

```
<button onclick="window.open('http://east-side-server:8000','_blank')">  
  Open East Side Server  
</button>
```

This works because:

- systemd keeps the service running
 - Dashboard doesn't need sudo
 - No security risk
-

OPTION B (Advanced): Dashboard Button → Start / Stop the Service

⚠ **This requires sudo permissions** — do this only if you understand the security implications.

1 Create a control script

```
sudo nano /usr/local/bin/east-side-control.sh
#!/bin/bash

case "$1" in
    start)
        systemctl start east-side-server
        ;;
    stop)
        systemctl stop east-side-server
        ;;
    restart)
        systemctl restart east-side-server
        ;;
    status)
        systemctl status east-side-server
        ;;
    *)
        echo "Usage: $0 {start|stop|restart|status}"
        exit 1
        ;;
esac
```

Make executable:

```
sudo chmod +x /usr/local/bin/east-side-control.sh
```

2 Allow dashboard user to control service (sudoers)

```
sudo visudo
```

Add **one line** (replace dashboarduser):

```
dashboarduser ALL=(root) NOPASSWD: /usr/bin/systemctl start east-side-server,
/usr/bin/systemctl stop east-side-server, /usr/bin/systemctl restart east-
side-server
```

3 Button example (local dashboard)

```
<form action="/cgi-bin/east-side-control.sh" method="post">
  <button name="action" value="restart">Restart East Side Server</button>
</form>
```

Or via curl:

```
sudo systemctl restart east-side-server
```

RECOMMENDED ARCHITECTURE (Best Practice)

For your use case:

- ✓ **systemd** → owns lifecycle
- ✓ **dashboard** → only opens URLs
- ✓ **no stop/start from UI** unless necessary

This mirrors **enterprise Linux services**.

QUICK COMMAND REFERENCE

```
sudo systemctl start east-side-server
sudo systemctl stop east-side-server
sudo systemctl restart east-side-server
sudo systemctl status east-side-server
sudo journalctl -u east-side-server -f
```

FINAL STATUS

- ✓ Auto-start at reboot
 - ✓ Resilient service
 - ✓ Clean logs
 - ✓ Dashboard-friendly
 - ✓ RHEL-native
 - ✓ Expandable later
-

If you want next

I can help you:

- Add a `/health` endpoint for dashboards
- Bind this to **port 80/443 via nginx**

- Add SELinux policy (important on RHEL!)
- Harden permissions
- Convert this into an RPM
- Add firewall rules cleanly

Just tell me the next step.